



TaxiTech Solutions · Taxímetro Digital

Prototipo en Python para calcular el importe de carreras de taxi en tiempo real

python 3.10+ gui customtkinter tests 62 OK historias 9/9 fase 3 Arquitectura y UX



Tarifas por defecto

Estado	Tarifa
Parado o velocidad < 20 km/h	0,02 €/s
En movimiento	0,05 €/s



Funcionalidades

- **CLI interactiva** — iniciar, cambiar estado, finalizar y encadenar carreras.
- **Cálculo continuo** por tramos según el estado del taxi.
- **Acceso protegido con contraseña** — PBKDF2-HMAC-SHA256 con 600.000 iteraciones y salt aleatorio por credencial; nunca se guarda en claro.
- **Interfaz gráfica moderna** (customtkinter) con tema oscuro elegante, botones grandes de colores y píldora de estado que cambia según la carrera.
- **Histórico persistente** en `data/historial_carreras.csv`.
- **Tarifas configurables** en `config/tarifas.json`.
- **Logs técnicos** en `logs/taximetro.log`.
- **62 tests automatizados** con `unittest`.



Uso rápido

CLI

```
python3 main.py
```

Primera ejecución: el programa pide crear una contraseña y confirmarla, sin mostrar los caracteres (mínimo 4 caracteres). **Ejecuciones siguientes:** pide la contraseña para entrar; si es incorrecta, la CLI termina con **código de salida 1**.

GUI

```
pip install -r requirements.txt # primera vez
python3 main.py --gui
```

La primera ejecución solicita crear y confirmar una contraseña; en las siguientes, la GUI requiere autenticarse antes de mostrar el taxímetro. Si la contraseña es incorrecta muestra un error y permite reintentar (se puede confirmar con la tecla `Enter`).

Interfaz con **tema oscuro elegante**: tarjetas redondeadas, importe grande de lectura inmediata y una **píldora de estado** que cambia de gris (sin carrera) a ámbar (parado) y verde (en movimiento). Los botones `Iniciar` (verde), `Parado` (ámbar), `Marcha` (cian), `Finalizar` (rojo) e `Historial` (gris) se actualizan cada 500 ms.

Comandos CLI

Comando	Alias	Descripción
<code>inicio</code>	<code>i</code>	Iniciar carrera
<code>parado</code>	<code>p</code>	Cambiar a taxi parado
<code>marcha</code>	<code>m</code>	Cambiar a taxi en movimiento
<code>ver</code>	<code>v</code>	Ver estado, tiempo e importe actual
<code>fin</code>	<code>f</code>	Finalizar carrera y guardar en el histórico
<code>historial</code>	<code>h</code>	Mostrar carreras guardadas
<code>salir</code>	<code>s</code>	Cerrar el programa



Cambiar tarifas

Las tarifas se cargan al iniciar el programa (CLI y GUI). Los cambios se aplican en la siguiente ejecución; no se recargan durante una carrera.

```
{
  "stopped_rate_per_second": 0.02,
  "moving_rate_per_second": 0.05
}
```

Si el fichero falta, tiene un JSON mal formado, le falta una clave o contiene una tarifa no numérica o negativa, el programa **no inicia** y muestra un error de configuración identificable.



Tests

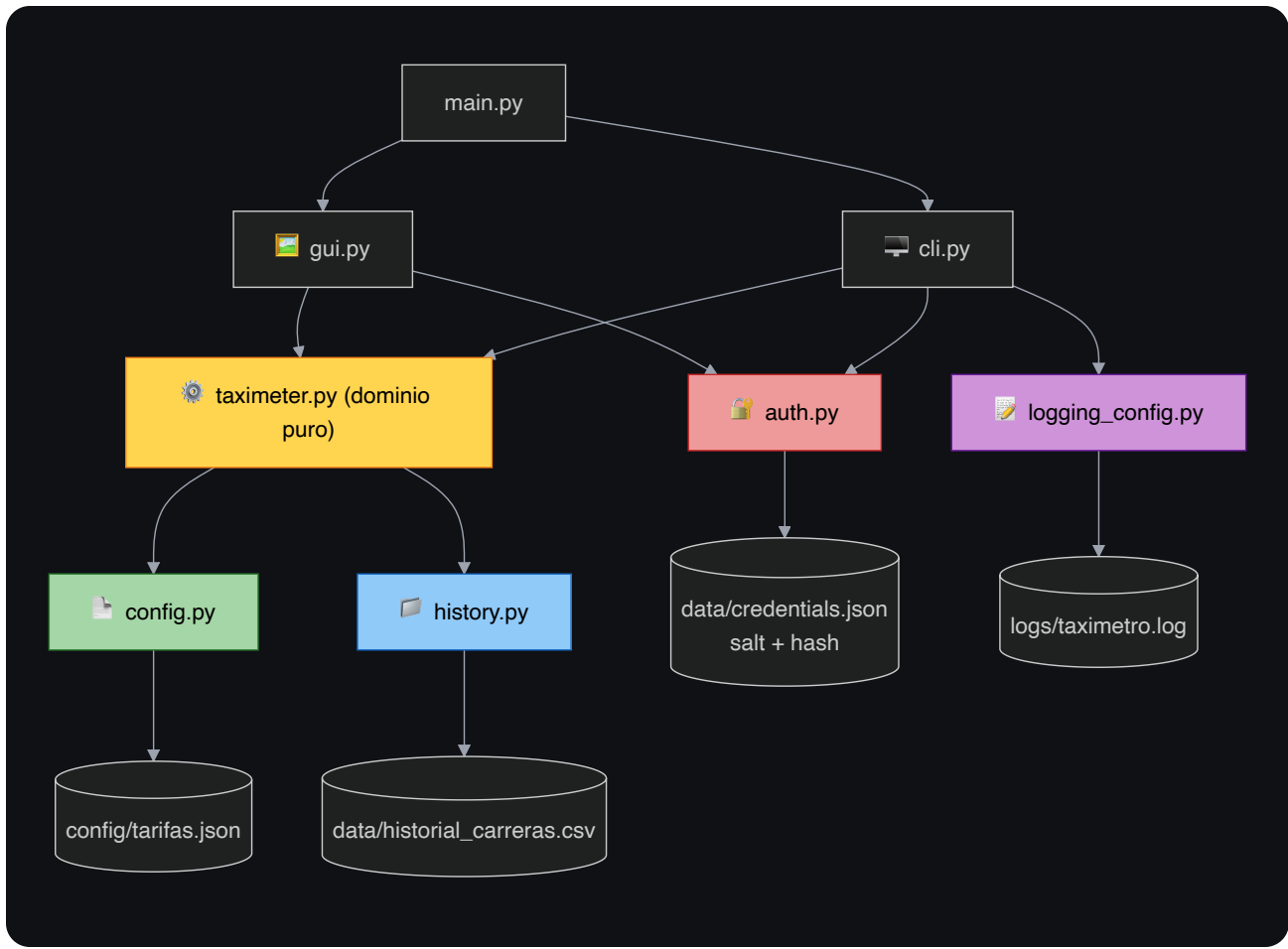
```
python3 -m unittest
```

62 tests OK — cubren dominio, configuración, autenticación, histórico, logging y las dos interfaces (CLI y GUI).



Arquitectura

Separación en tres capas: **presentación** (CLI/GUI), **dominio** puro sin E/S y **infraestructura** (ficheros y logs).



Claves de diseño:

- 🧠 **Taximeter** es **código de dominio puro**: no hace E/S y recibe las tarifas y un **reloj inyectado**, lo que permite simular carreras de 30 s en microsegundos durante los tests.
- 🚫 **Sin acoplamiento a archivos**: cada componente recibe su ruta por parámetro con un valor por defecto.
- 🔁 **CLI y GUI comparten el mismo dominio**: ambas usan **Taximeter**, **TripHistory** y **PasswordAuth**.
- 🛡️ **Validación exhaustiva** de la configuración: tipos, valores no negativos, números finitos y JSON mal formado.

📁 Estructura

```
.
├── config/tarifas.json      # tarifas configurables
├── data/                   # generado en runtime (git ignored)
│   ├── credentials.json    # salt + hash de la contraseña
```

```
| └─ historial_carreras.csv
|─ docs/github-project-tasks.md
|─ logs/taximetro.log          # generado en runtime (git ignored)
|─ main.py
|─ requirements.txt           # customtkinter
|─ scripts/sync_github_project_tasks.sh
|─ taximeter/
|   └─ auth.py                # autenticación PBKDF2
|   └─ cli.py                 # interfaz de línea de comandos
|   └─ config.py              # carga y validación de tarifas
|   └─ gui.py                 # interfaz gráfica (customtkinter, tema oscuro)
|   └─ history.py             # histórico CSV
|   └─ logging_config.py      # configuración de logs
|   └─ taximeter.py           # núcleo de cálculo (dominio puro)
└─ tests/
    └─ test_auth.py
    └─ test_cli.py
    └─ test_config.py
    └─ test_gui.py
    └─ test_history.py
    └─ test_history_interfaces.py
    └─ test_logging.py
    └─ test_taximeter.py
```

Historias de usuario

ID	Historia	Estado
US-01	Iniciar una carrera desde CLI	✓
US-02	Cambiar estado entre parado y movimiento	✓
US-03	Finalizar carrera y mostrar total	✓
US-04	Encadenar varias carreras sin cerrar el programa	✓
US-05	Guardar y consultar histórico de carreras	✓
US-06	Registrar logs de operación y errores	✓
US-07	Cargar tarifas desde fichero de configuración	✓

ID	Historia	Estado
US-08	Proteger acceso con contraseña segura	✓
US-09	Crear interfaz gráfica con botones grandes	✓

Requisitos y evolución

- **Requisitos:** Python 3.10 o superior y `pip install -r requirements.txt` (solo `customtkinter` para la GUI; la CLI funciona sin instalar nada).
- La **fase 4** queda documentada como evolución futura porque requiere API, base de datos y despliegue.